

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192521344
Name	NANDHINI SRI V

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)*

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I NANDHINI SRI V(192521344) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1.Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

1. Introduction

A media platform handles a large volume of videos, images, audio files, subtitles, thumbnails, and other digital content. The storage system must provide continuous data availability while also controlling storage costs.

Hadoop Distributed File System (HDFS) is suitable for this requirement because it stores large files by dividing them into blocks and distributing those blocks across multiple DataNodes. The proposed solution is designed according to the three given constraints: **availability above 99.9%, replication factor not greater than 3, and cost-efficient storage growth.**

2. Constraints

The main constraints of the media platform are:

Constraint	Required Condition
Data availability	More than 99.9%
Replication factor	Maximum 3
Storage growth	Must be cost efficient
Fault tolerance	Data must remain available during node failure
Data volume	Suitable for continuously growing media data

3. HDFS Storage Design

The proposed system uses an HDFS cluster with the following major components:

NameNode: Maintains information about files, directories, permissions, and the locations of blocks.

Standby NameNode: Provides backup for the Active NameNode and helps maintain service availability if the Active NameNode fails.

DataNodes: Store the actual media files as HDFS blocks.

JournalNodes: Maintain shared information required for NameNode high availability.

The media application communicates with HDFS through an HDFS client. Large media files are divided into blocks and stored across multiple DataNodes.

4. Replication Factor

The replication factor is configured as **3**, which satisfies the condition that it must not exceed 3.

For example, if a video file is divided into several blocks, each block can have three copies stored on different DataNodes.

This provides fault tolerance because the failure of one DataNode does not result in the loss of the media block. Another DataNode can provide the required copy.

A replication factor of 3 is selected because:

- It provides good fault tolerance.
- It protects against individual DataNode failures.
- It does not exceed the maximum allowed replication factor.
- It avoids the additional storage cost of maintaining more than three copies.

5. Achieving More Than 99.9% Availability

High availability can be achieved using several HDFS features.

NameNode High Availability

The system uses an **Active NameNode and a Standby NameNode**. If the Active NameNode fails, the Standby NameNode can take over the metadata service.

This prevents the NameNode from becoming a single point of failure.

Data Replication

Each important HDFS block has three replicas. If one DataNode becomes unavailable, the application can access another replica from a healthy DataNode.

Rack-Aware Storage

HDFS can place replicas across different racks. This reduces the possibility of losing all replicas if one rack experiences a power or network failure.

Automatic Recovery

HDFS monitors DataNodes using heartbeats and block reports. When a DataNode fails, HDFS identifies under-replicated blocks and creates replacement replicas on healthy DataNodes.

These features together help the platform maintain the required **greater than 99.9% availability**.

6. Cost-Efficient Storage Growth

Media content can grow continuously, so the storage system should be expandable without replacing the existing infrastructure.

HDFS supports **horizontal scaling**, where additional DataNodes can be added to the cluster when more storage capacity is required.

For example, the platform can initially operate with a small number of DataNodes. As the number of videos and images increases, additional DataNodes can be added.

This approach is cost efficient because:

- Existing servers continue to be used.
- Storage capacity can be increased gradually.
- New hardware is purchased only when required.
- The workload can be distributed across additional DataNodes.
- The entire storage system does not need to be replaced.

Frequently accessed media can also be kept on suitable high-performance storage, while older or less frequently accessed content can be moved to lower-cost storage according to the organization's storage policy.

7. Working of the Proposed Solution

1. The media application uploads a video or other media file to HDFS.
2. HDFS divides the large file into blocks.
3. The NameNode maintains the metadata and block-location information.
4. The blocks are distributed among DataNodes.
5. Each block is replicated three times.

6. Replicas are placed on different DataNodes to improve fault tolerance.
7. The NameNode continuously monitors the health of DataNodes.
8. If a DataNode fails, another replica is used to serve the data.
9. HDFS creates a replacement replica to restore the required replication level.
10. When storage requirements increase, new DataNodes are added to the cluster.

8. Constraint Satisfaction

Constraint	Proposed Solution
Availability above 99.9%	Active/Standby NameNodes, replication and automatic recovery
Replication factor cannot exceed 3	Replication factor fixed at 3
Cost-efficient growth	Add DataNodes gradually using horizontal scaling
DataNode failure	Access data from another replica
Rack failure	Distribute replicas using rack-aware placement
Increasing media volume	Expand the HDFS cluster by adding DataNodes

9. Justification

The proposed HDFS solution is suitable for a media platform because HDFS is designed to handle large files and large-scale distributed storage. The replication factor of 3 provides a suitable balance between reliability and storage consumption.

NameNode high availability prevents metadata failure from stopping the complete storage service. Data replication and automatic re-replication protect media data against DataNode failures. Rack-aware placement provides additional protection against infrastructure-level failures.

The scale-out approach also makes the solution cost efficient because storage capacity can be increased gradually instead of purchasing a completely new storage system.

10. Conclusion

The proposed HDFS storage solution satisfies all the given constraints. **NameNode high availability, DataNode replication, rack-aware placement, and automatic recovery** help achieve availability above

99.9%. A **replication factor of 3** provides fault tolerance without exceeding the specified limit.

Horizontal scaling through additional DataNodes allows the media platform to increase its storage capacity in a cost-efficient manner.

Therefore, HDFS provides a reliable, scalable, fault-tolerant, and cost-efficient storage solution for the media platform.

2.A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

1. Introduction

A large application generates log data containing information such as user requests, errors, response times, IP addresses, and access timestamps. The organization needs to process these logs within a **20-minute batch window** while using a limited number of cluster nodes.

MapReduce can process the logs in parallel by dividing the input into blocks and distributing the processing tasks across available nodes.

2. Constraints

Constraint	Requirement
Processing time	Complete within 20 minutes
Cluster resources	Limited number of nodes
Data size	Large log dataset
Processing method	MapReduce
Output	Useful summary of log information

3. Proposed Job Design

The MapReduce job is divided into two major stages:

Mapper stage: Reads log records and extracts useful information.

Reducer stage: Groups related information and produces final results.

For example, if the requirement is to calculate the number of requests and errors for each service, the Mapper can generate key-value pairs such as:

ServiceA → 1

ServiceA → 1

ServiceB → 1

The Reducer groups the same keys and calculates their totals.

4. Input Data Preparation

The log files are stored in HDFS. Large files are divided into blocks. Each block can be processed by a separate Mapper task.

Input data should be organized into reasonably sized files to avoid excessive task-management overhead. The input format should allow Hadoop to split the logs efficiently.

5. Mapper Task

The Mapper reads each log record and extracts required fields.

For example:

2026-08-27 ServiceA ERROR

The Mapper may produce:

ServiceA, ERROR → 1

For a successful request:

ServiceA, SUCCESS → 1

The Mapper performs filtering and basic processing before sending the intermediate results to the Reducer.

6. Shuffle and Sort

After the Mapper stage, Hadoop performs shuffle and sort.

During this stage:

- Similar keys are grouped together.

- Intermediate data is transferred to the appropriate Reducer.
- Keys are sorted before reaching the Reducer.
- Unnecessary data can be reduced using a Combiner.

A **Combiner** can perform local aggregation on each Mapper node. This reduces the amount of intermediate data transferred across the network.

7. Reducer Task

The Reducer receives grouped records and calculates final values.

For example:

ServiceA, ERROR $\rightarrow$ [1,1,1,1]

The Reducer calculates:

ServiceA, ERROR $\rightarrow$ 4

Similarly, successful requests and other required metrics can be calculated.

8. Task Distribution with Limited Nodes

Since the cluster has limited nodes, the job should use the available resources efficiently.

The input is divided into multiple splits, and Mapper tasks are assigned to available nodes. Hadoop attempts to use **data locality**, meaning that computation is performed close to where the input data is stored.

The number of Reducers should be selected according to the available CPU and memory instead of creating too many tasks.

If only a few nodes are available, the system can process multiple task waves rather than requiring additional machines.

9. Meeting the 20-Minute Requirement

The following techniques help complete the batch within 20 minutes:

- Use appropriate input splits.
- Enable data-local processing.

- Use a Combiner to reduce network traffic.
- Filter unnecessary records during the Mapper stage.
- Avoid unnecessary Reducer operations.
- Allocate available CPU and memory effectively.
- Use compression for intermediate MapReduce data where suitable.
- Monitor slow or failed tasks.
- Avoid excessive numbers of very small input files.

The total execution should be monitored so that Map, Shuffle, and Reduce stages together remain within the 20-minute window.

10. Constraint Satisfaction

Constraint	Solution
20-minute processing window	Parallel MapReduce processing and optimized tasks
Limited nodes	Efficient resource allocation and multiple task waves
Large log data	HDFS block-based input processing
Network overhead	Combiner and data-local processing
Failed tasks	Hadoop automatically re-executes failed tasks

11. Justification

MapReduce is suitable because it allows a large log dataset to be divided into smaller processing tasks. Multiple Mapper tasks can execute in parallel using the limited cluster resources.

Data locality reduces network transfer, while the Combiner reduces intermediate data before the Shuffle stage. Hadoop's task retry mechanism also improves reliability without requiring manual intervention.

12. Conclusion

The proposed MapReduce design can efficiently process large log datasets within the 20-minute batch requirement by using parallel Mapper tasks, controlled Reducer tasks, data locality, and Combiners. The design makes effective use of limited cluster nodes while maintaining reliable processing.

3.A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under constraints of fairness, priority for ETL, and recovery from node failure.

1. Introduction

An organization uses a YARN-managed Hadoop cluster to execute three major workloads: ETL jobs, reporting jobs, and machine learning jobs. These workloads have different resource requirements and priorities.

A suitable scheduler policy must provide **fairness**, give **priority to ETL jobs**, and recover properly when a node fails.

2. Constraints

Constraint	Requirement
Fairness	Resources should be shared among workloads
ETL priority	ETL jobs should receive higher priority
Reporting	Should receive reasonable resources
Machine learning	Should receive resources without starving other jobs
Node failure	Jobs must recover automatically

3. Proposed YARN Scheduling Policy

A **Capacity Scheduler** can be used to divide the cluster into separate queues.

Three logical queues can be created:

- **ETL Queue**
- **Reporting Queue**
- **Machine Learning Queue**

The ETL queue receives a higher guaranteed capacity because ETL processing is given priority.

However, reporting and machine learning queues should also receive minimum resources so that they are not permanently blocked.

4. ETL Priority

ETL jobs are important because they prepare and transform data for downstream analytics. Therefore, the ETL queue can receive a higher capacity and higher scheduling priority.

For example:

Queue	Priority	Suggested Capacity
ETL	High	50%
Reporting	Medium	25%
Machine Learning	Medium	25%

These values can be adjusted according to actual workload requirements.

When the ETL queue is not using its full capacity, unused resources can be temporarily allocated to other queues.

5. Fairness

Fairness is maintained by preventing one workload from continuously consuming all cluster resources.

Reporting and machine learning applications receive their own resource allocation. Within each queue, jobs can be scheduled fairly according to their resource requirements.

This prevents a large machine learning job from stopping reporting tasks completely.

6. Resource Management

YARN consists mainly of:

ResourceManager: Controls cluster-level resource allocation and scheduling.

NodeManager: Manages containers and resources on individual worker nodes.

ApplicationMaster: Manages the execution of an individual application.

The ResourceManager assigns resources to applications based on the configured scheduler policy.

7. Node Failure Recovery

If a worker node fails, its NodeManager stops responding to the ResourceManager.

YARN detects the failure using heartbeat monitoring. Containers running on the failed node are considered lost.

The ApplicationMaster can request new containers from healthy nodes and restart the required tasks.

This allows the workload to continue without manually restarting the entire cluster.

8. Preventing Starvation

Although ETL receives higher priority, reporting and machine learning jobs should not be completely denied resources.

Minimum queue capacities and fair scheduling help prevent starvation.

If the ETL workload temporarily becomes smaller, unused capacity can be shared with other queues.

When ETL demand increases again, its higher priority allows it to obtain resources.

9. Proposed Policy

The scheduling policy can follow these rules:

1. Give ETL jobs the highest priority.
2. Reserve minimum resources for reporting.
3. Reserve minimum resources for machine learning.
4. Allow unused resources to be shared.
5. Use queue limits to prevent a single workload from consuming the entire cluster.
6. Detect failed nodes through NodeManager heartbeats.
7. Restart failed containers on healthy nodes.
8. Monitor queue utilization and modify capacity when workload patterns change.

10. Constraint Satisfaction

Constraint	Solution
Fairness	Separate queues and controlled resource sharing
ETL priority	Higher capacity and scheduling priority

Constraint	Solution
Reporting	Guaranteed queue capacity
Machine learning	Dedicated resource allocation
Node failure	YARN detects failures and restarts containers
Resource utilization	Unused capacity can be shared

11. Justification

The proposed Capacity Scheduler policy is suitable because it allows different workloads to receive controlled portions of the cluster resources. ETL receives priority without completely blocking reporting and machine learning.

YARN's ResourceManager and NodeManagers provide centralized resource management and failure detection. Failed tasks can be restarted on healthy nodes, improving cluster reliability.

12. Conclusion

The proposed YARN scheduling policy provides a balance between priority and fairness. ETL receives higher priority, while reporting and machine learning jobs receive guaranteed resources. Node failure is handled through YARN's monitoring and task recovery mechanisms. Therefore, the policy provides efficient and reliable resource management for the Hadoop cluster.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

1. Introduction

An enterprise may have data stored in different systems such as Storage Area Networks (SAN), Network Attached Storage (NAS), and operational databases. To perform big data analytics, this information must be transferred into a centralized Hadoop-based platform.

The proposed architecture focuses on reliable backup, minimum duplication, controlled ingestion, and consistent data management.

2. Constraints

Constraint	Requirement
Data sources	SAN, NAS, and operational databases
Backup reliability	Data must be recoverable
Duplication	Avoid unnecessary duplicate data
Integration	Data should reach the Hadoop platform efficiently
Scalability	System should support increasing data volume

3. Proposed Components

The solution uses:

- **Apache NiFi or ETL tools** for data ingestion.
- **HDFS** for distributed storage.
- **Hive** for structured data analysis.
- **Metadata management** for tracking datasets.
- **Checksum or record validation** for duplicate detection.
- **Backup storage** for recovery.
- **Monitoring** for ingestion failures.

4. SAN Data Ingestion

SAN contains enterprise files and storage volumes. Required data can be extracted using appropriate file-access or ETL mechanisms.

Only the required datasets should be copied into the Hadoop platform. Metadata such as source, timestamp, file size, and checksum can be recorded.

This prevents repeated ingestion of the same files.

5. NAS Data Ingestion

NAS typically contains shared files such as documents, media, and business records.

The ingestion process reads the required files and transfers them to HDFS. A checksum can be calculated for each file.

If the checksum already exists in the metadata repository, the file does not need to be copied again.

6. Operational Database Ingestion

Operational databases contain structured information such as customer, transaction, order, and employee records.

ETL tools or Apache NiFi can extract database records using JDBC or other supported database connectors.

For large databases, **incremental ingestion** is preferred. Instead of copying the entire database repeatedly, only newly inserted or modified records are transferred.

7. Minimizing Duplication

Data duplication can be controlled using:

- File checksums.
- Unique record identifiers.
- Source-system identifiers.
- Incremental extraction.
- Metadata tracking.
- Duplicate validation before storage.

For database records, a primary key or business key can be used to identify existing records.

For files, checksum comparison can determine whether two files have identical content.

8. Backup Reliability

HDFS provides fault tolerance through data replication. A replication factor can be selected according to the organization's reliability requirements.

Important datasets can also be backed up to a separate storage location. Backup copies should be monitored and periodically tested to ensure that data can actually be restored.

Backup metadata should include:

- Dataset name
- Source system
- Backup date
- Version
- Storage location
- Validation status

9. Data Organization

After ingestion, the data can be organized in HDFS according to source and data type.

For example:

- SAN data
- NAS data
- Database data
- Historical data
- Processed data

Access permissions should be applied according to the sensitivity of the data.

10. Constraint Satisfaction

Constraint	Solution
SAN ingestion	Controlled file-based ETL/NiFi ingestion

Constraint	Solution
NAS ingestion	File ingestion with metadata and checksum
Database ingestion	Incremental database extraction
Backup reliability	HDFS replication and separate backup copies
Minimal duplication	Checksums, keys, and metadata tracking
Scalability	Distributed HDFS storage

11. Justification

Using an ingestion layer such as Apache NiFi or an ETL platform provides centralized control over data movement. It can monitor transfers, detect failures, and maintain information about the source of each dataset.

Incremental database extraction reduces unnecessary data transfer. Checksum and key-based validation reduce duplicate storage. HDFS provides scalable distributed storage and fault tolerance.

12. Conclusion

The proposed architecture integrates SAN, NAS, and operational database sources into a Hadoop-based platform using controlled ingestion. Checksums, metadata, unique identifiers, and incremental extraction minimize duplication. HDFS replication and separate backup mechanisms improve reliability. Therefore, the design provides a scalable and dependable enterprise data ingestion solution.

5.Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

1. Introduction

An enterprise requires an end-to-end big data platform that can securely move data from different sources, store it reliably, process it efficiently, and integrate it with ETL pipelines.

The proposed solution combines **Apache NiFi, HDFS, YARN, MapReduce, and security mechanisms** to satisfy the requirements of secure data movement, high availability, and ETL integration.

2. Constraints

Constraint	Requirement
------------	-------------

Data movement	Must be secure
---------------	----------------

Availability	High availability is required
--------------	-------------------------------

Processing	Hadoop ecosystem should be used
------------	---------------------------------

ETL integration	Apache NiFi or ETL pipeline must be supported
-----------------	---

Data storage	Reliable and scalable
--------------	-----------------------

Failure handling	System should continue after component failure
------------------	--

3. Data Ingestion Using Apache NiFi

Apache NiFi can be used as the data ingestion layer.

It can collect data from:

- Databases
- Application logs
- Files
- Network systems
- External data sources

NiFi provides processors for receiving, transforming, routing, and transferring data.

It also provides flow management and monitoring, which makes it suitable for enterprise data movement.

4. Secure Data Movement

Secure communication should be enabled between data sources, NiFi, and Hadoop components.

The design can use:

- TLS encryption for data in transit.
- Authentication for users and services.
- Authorization using access permissions.
- Secure credentials management.
- Auditing of important data operations.

Sensitive information should not be transferred through unencrypted communication channels.

5. HDFS Storage

After ingestion, data is stored in HDFS.

HDFS divides large datasets into blocks and distributes them across DataNodes. Replication provides protection against DataNode failure.

For important datasets, a suitable replication factor can be configured while considering storage requirements.

HDFS can also be configured with NameNode High Availability using Active and Standby NameNodes.

6. High Availability

High availability is maintained at different levels.

NameNode High Availability

An Active NameNode and Standby NameNode are used. If the Active NameNode fails, the Standby NameNode can take over.

DataNode Fault Tolerance

Data blocks are replicated across multiple DataNodes. If one DataNode fails, another replica can provide the required data.

NiFi Reliability

NiFi maintains information about data flow and can handle failed transfers using queues and controlled retries.

Together, these mechanisms reduce the effect of individual component failures.

7. Processing Using YARN and MapReduce

YARN manages cluster resources and allocates containers to applications.

MapReduce can process large datasets stored in HDFS.

The Mapper processes input records and generates intermediate key-value pairs. The Reducer combines related values and produces final results.

YARN ensures that processing applications receive the required CPU and memory resources.

8. ETL Integration

NiFi can perform initial data collection, routing, filtering, and transformation before storing data in HDFS.

For more complex transformations, dedicated ETL jobs can be executed using Hadoop processing frameworks.

A typical process is:

1. Data is collected from source systems.
2. NiFi validates and routes the data.
3. Secure communication transfers the data.
4. Data is stored in HDFS.
5. YARN allocates processing resources.
6. MapReduce or other Hadoop jobs process the data.
7. Processed data is stored for analysis.
8. Monitoring systems track the complete pipeline.

9. Security and Access Control

Security should be applied at multiple levels.

Authentication verifies the identity of users and services. Authorization ensures that users can access only permitted datasets.

HDFS permissions and enterprise security mechanisms can be used to protect stored data.

Data transfers should use encryption, and important activities should be recorded through audit logs.

10. Failure Recovery

If a data transfer fails, NiFi can retry or route the failed flow for further handling.

If a DataNode fails, HDFS uses another replica. HDFS can recreate missing replicas to restore the required replication level.

If the Active NameNode fails, the Standby NameNode can provide continued metadata service.

If a MapReduce task fails, Hadoop can re-execute the task on an available node.

Thus, failures at different stages can be handled without stopping the complete system.

11. Constraint Satisfaction

Constraint	Proposed Solution
Secure data movement	TLS, authentication, authorization and auditing
High availability	Active/Standby NameNodes and replicated HDFS blocks
NiFi/ETL integration	NiFi ingestion and ETL processing pipeline
Scalable storage	HDFS distributed storage
Processing	YARN and MapReduce
Failure recovery	NiFi retries, HDFS replication and MapReduce task re-execution

12. Justification

The proposed solution combines the strengths of different Hadoop ecosystem components. NiFi provides controlled data ingestion and flow management. HDFS provides distributed and fault-tolerant storage. YARN manages cluster resources, while MapReduce performs distributed data processing.

Security mechanisms protect data during movement and access. NameNode High Availability and HDFS replication reduce service interruptions. Failure recovery at the ingestion, storage, and processing levels improves overall reliability.

13. Conclusion

The proposed end-to-end Hadoop ecosystem satisfies the requirements for secure data movement, high availability, and ETL integration. Apache NiFi provides controlled ingestion, HDFS provides reliable distributed storage, YARN manages resources, and MapReduce performs large-scale processing.

By combining encryption, access control, replication, high-availability configuration, monitoring, and recovery mechanisms, the system provides a **secure, scalable, reliable, and integrated big data processing platform**.